Few-Shot Bearing Fault Diagnosis Based on Model-Agnostic Meta-Learning: A Reproduction Study

Samarth Sharma | samarth.011@nmims.in

Abstract—This study reproduces the central methodology of a model-agnostic meta-learning (MAML) approach to few-shot bearing fault diagnosis. An independently written PyTorch implementation combines a four-block convolutional neural network, full second-order meta-gradients, and learnable inner-loop learning rates. Thirty real CWRU recordings produce 1,771 paired vibration windows; each window contains 2,048 samples from each of two sensors and is represented as a 64 × 64 input. Meta-training and meta-test classes are disjoint. Three independent seeds yield 94.05 ± 1.20% accuracy for 3-way 5-shot evaluation and 58.03 ± 5.30% for 5-way 5-shot evaluation. The deviations are sample standard deviations across seed-level accuracies. The implementation uses 100 meta-iterations and one task per meta-batch, substantially below the reference training budget. Window construction, normalization, and evaluation assumptions are explicitly documented. Saved predictions, checkpoints, manifests, and an executed notebook support verification. These results demonstrate a practical reproduction of the learning mechanism; they do not establish exact replication or improvement over the original experimental protocol.

Index Terms—Bearing fault diagnosis, few-shot learning, model-agnostic meta-learning, CWRU, reproducibility.

I. INTRODUCTION

Bearings are central to rotating machinery. Defects in the rolling elements or raceways change the vibration response of the assembly. A supervised classifier ordinarily requires many labeled examples for each condition, including operating loads and defect severities that may be expensive to reproduce. A few-shot classifier instead receives a small labeled support set for a new classification task. The relevant question is whether knowledge learned from other fault classes helps it adapt quickly.

Here, N-way K-shot means N classes in an episode and K labeled support examples for each class. Query examples are different windows from those same classes and are used to measure adaptation. At meta-test time, these classes were absent from the initialization training. This distinction is essential: randomly dividing windows from every class into ordinary training and testing sets would measure a different problem. Both requested experiments are therefore constructed with disjoint class sets.

The reference is Shen Zhang, Fei Ye, Bingnan Wang and Thomas G. Habetler, Few-Shot Bearing Fault Diagnosis Based on Model-Agnostic Meta-Learning, arXiv:2007.12851v4 (2021) [1]. The implementation uses its CWRU class definitions, paired signal input and convolutional backbone. It implements both fixed and learnable inner learning rates; the reported experiments use the learnable variant. The article also studies Paderborn artificial-to-real transfer. That additional dataset, optimizer sweeps and the full set of fault splits are outside this submission. Table V is the reference for the two requested CWRU comparisons. No external implementation was copied into this project.

Related work provides context for interpreting this reproduction. Smith and Randall [2] present a diagnostic benchmark analysis of the CWRU recordings. In few-shot learning, prototypical networks [3] classify examples by their distance to class prototypes in a learned embedding space, whereas this study adapts model parameters through gradient updates. Antoniou et al. [4] discuss MAML training sensitivity and propose improvements to stability and generalization. These studies motivate careful reporting of optimization and evaluation choices; their methods are not additional completed baselines in this project.

The objective is to evaluate whether the essential method can be implemented, debugged, and audited within a limited computational budget. The contribution is an empirical reproduction with explicit decisions for underspecified details. It is not a new diagnostic algorithm. The two selected experiments retain the reference class ordering and evaluate adaptation to held-out classes, while the analysis distinguishes source-reported values from measurements obtained by the present implementation.

The remainder follows the structure of the reference article. Section II describes the meta-learning objective and learned-rate adaptation. Section III presents the dataset, preprocessing and classifier. Section IV reports the experimental setup, actual results, comparison and limitations. Section V concludes the reproduction study.

II. PRINCIPLE OF MODEL-AGNOSTIC META-LEARNING

A. Episodic Few-Shot Learning

Following MAML [5], a task contains a support set S and a query set Q over the same N classes. The support set supplies K labels per class. The query set contains separate examples. During meta-training, tasks are sampled from the training-class pool. During testing, tasks are sampled from a disjoint class pool. Local labels 0 through N−1 are assigned within each episode; this assignment need not coincide with the physical class IDs.

For a classifier with parameters θ, support adaptation minimizes cross-entropy. Query loss then assesses whether that update transfers to different examples of the current task. A small support loss alone is insufficient evidence of useful

adaptation, because it may reflect fitting the labeled examples without discriminating query examples.

B. Inner Adaptation and the Meta-Objective

$$\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{S_i}(\theta) \tag{1}$$

Equation (1) gives one inner gradient step for task i. The initial parameters θ are shared across tasks; the adapted parameters are task-specific. The meta-objective averages query losses at these adapted parameters over B training tasks:

$$\mathcal{J}(\theta) = \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}_{Q_i}(\theta_i') \tag{2}$$

$$\theta \leftarrow \theta - \beta \nabla_\theta \mathcal{J}(\theta) \tag{3}$$

Equation (3) expresses the meta-gradient descent direction. The actual implementation uses Adam [6], which applies moment estimates to this gradient. Full second-order differentiation retains the dependence of the inner gradient on θ. In PyTorch [7], create_graph=True retains that computation graph, while functional parameter substitution evaluates adapted weights without modifying the shared initialization in place.

C. Learnable Inner-Loop Learning Rates

$$\theta_{i,\ell}' = \theta_\ell - \alpha_{\ell,0} \nabla_{\theta_\ell} \mathcal{L}_{S_i}(\theta) \tag{4}$$

The scalar α in (1) is replaced by a learned rate for each parameterized layer and inner step, as in the learned-rate variant examined by Zhang et al. [1]. The submitted runs use one step, so (4) shows the step-0 rate for layer ℓ. Weights and biases in the same layer share that rate. All rates start at 0.01 and receive outer gradients together with θ. There are nine additional scalars: four convolutional, four normalization and one head rate.

Learning an update rule is also studied by Meta-SGD [8], which learns initialization, update direction and learning rate. This reproduction follows the bearing paper's layer-shared rate formulation; it does not claim to reproduce Meta-SGD or all MAML++ modifications [4].

D. Test-Time Adaptation

Batch-normalization statistics are computed from support examples at the adapted weights and reused for queries. This makes evaluation non-transductive: changing other query examples cannot change a given query prediction. All CNN parameters adapt; the initialization resets for every test task. Query labels have no role in this adaptation.

No outer update is performed at test time. Each episode begins at the same saved initialization and adapts using only its support labels. Query labels are used for scoring. Predictions are mapped back to physical classes before confusion matrices are aggregated, preventing unrelated episode-local label assignments from being combined incorrectly.

III. FEW-SHOT BEARING FAULT DIAGNOSTIC FRAMEWORK

A. CWRU Dataset and Class Definitions

Thirty original MATLAB files were downloaded from the official university pages [9]-[11]. The data cover ten classes at 1, 2 and 3 HP; load is not itself the class label. Fault sizes are 0.007, 0.014 and 0.021 inches. Outer-race records use the centered 6-o'clock position. The table below is computed from the loaded recordings after preprocessing, not copied from the article.

TABLE I
SELECTED CWRU CLASSES AND PROCESSED WINDOWS

ID	Condition	Files	Windows
1	Healthy	3	177
2	Ball 007	3	177
3	Ball 014	3	177
4	Ball 021	3	177
5	IR 007	3	178
6	IR 014	3	177
7	IR 021	3	177
8	OR 007	3	177
9	OR 014	3	177
10	OR 021	3	177

The loader produces 1,771 paired, non-overlapping windows in total. A source manifest stores the exact channel keys, original lengths, assumed rates and SHA-256 hash of each file. A window manifest records recording ID, class, load and sample boundaries. A separate manifest for each run records the nine chosen training examples and the test support/query pools. Together with seeds and saved configurations, these records make the experiments auditable.

Healthy files 98, 99 and 100 are treated as 48 kHz recordings and both channels are decimated to 12 kHz using a polyphase anti-aliasing filter. The official baseline table does not explicitly document each channel's rate, so this choice remains an assumption. Fault files are selected from the 12k drive-end page. Both loaded channels must have equal lengths. In 99.mat the X099 channels are selected explicitly because that official file also embeds X098 channels; choosing the first matching key would silently duplicate another recording.

Table I contains measurements from the loaded recordings. All three loads contribute windows to each class pool. For the 3-way experiment, training classes are 1–7 and testing classes are 8–10. For the 5-way experiment, training classes are 1–5 and testing classes are 6–10. These are independent experiments: a class can have a different role between experiments, but never belongs to both roles within one experiment.

B. Signal Segmentation and Normalization

For each recording, the drive-end and fan-end channels are segmented at the same offsets into 2,048-point windows with stride 2,048. The two windows are concatenated in DE-then-FE order. Their 4,096 values are centered and divided by their joint standard deviation, using only that example. The vector is reshaped in row-major order into a single-channel 64 x 64 array. This is a representation of time-domain values, not a spectrogram or an FFT. No fitted normalization statistic is shared between meta-training and testing.

The paper reports more segments per class but does not give an exact stride or scaling rule. Non-overlapping windows are a conservative, reproducible choice and explicitly differ from that reported pool. At meta-test time the first half of each recording supplies support windows; the second half supplies query windows. One complete window between the halves is purged. This prevents raw sample overlap between the two roles across episodes. Windows can still be statistically correlated because they share a bearing and acquisition condition.

$$\tilde{z}_j = \frac{z_j - \mu_z}{\max(\sigma_z, 10^{-8})}, \quad j = 1, \ldots, 4096 \tag{5}$$

In (5), z is the concatenated DE/FE vector and its mean and population standard deviation are computed within that single example. The denominator is bounded for numerical stability. The resulting array has unit population variance unless the original signal is constant. Using joint normalization retains relative amplitudes between the two sensors within a window, while removing a common scale. This design choice is not asserted to be the normalization used in the original study.

C. CNN Architecture and Model Implementation

TABLE II
CONVOLUTIONAL CLASSIFIER ARCHITECTURE

Stage	Operation	Output
Input	Two-sensor representation	1 × 64 × 64
Block 1	Conv–BN–ReLU–Pool	64 × 32 × 32
Block 2	Conv–BN–ReLU–Pool	64 × 16 × 16
Block 3	Conv–BN–ReLU–Pool	64 × 8 × 8
Block 4	Conv–BN–ReLU–Pool	64 × 4 × 4
Head	Flatten 1024; linear	N logits

Each block uses 64 convolutional filters of size 3 × 3, batch normalization, a ReLU nonlinearity, and 2 × 2 max-pooling, following the backbone specification in [1]. Table II shows the resulting dimensions. Stride-one, same-padded convolutions preserve spatial size before pooling. The flatten-and-linear head is an explicit resolution of an underspecified architectural detail; no modern replacement backbone is introduced.

The CNN has 115,011 trainable parameters for three classes and 117,061 for five classes. Learnable-rate MAML adds nine scalars for one inner step: four convolution rates, four normalization rates and one classifier rate. Convolutions use stride one and padding one. A layer's weight and bias share its rate. Padding, head layout and rate sharing are documented implementation choices. Cross-entropy receives logits and applies log-softmax internally.

Batch normalization was introduced by Ioffe and Szegedy [12] to normalize intermediate activations during network training. Here, reusing support-derived statistics for queries is an explicit episodic evaluation choice, rather than a claim about the original batch-normalization paper's test protocol.

D. Reproducibility and Leakage Controls

Nine examples are drawn without replacement for every meta-training class using the run seed. A 5-shot training episode splits these into five support and four query examples. Support and query indices cannot overlap inside an episode. At evaluation, fixed temporal role pools ensure that a window is not a support example in one episode and a query example in another. Entire source files are exclusive to the training or test side because the class split precedes task sampling.

The dataset is not included in the repository or submission archive. A one-command optional downloader and a manual table of all file IDs are provided. Manual setup consists of downloading the indicated files, preserving their numeric filenames, placing them in a local directory, and changing DATASET_PATH. There is no synthetic-data fallback. Missing files or ambiguous channels raise an error rather than silently reducing the experiment.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

TABLE III
REPRODUCTION HYPERPARAMETERS AND PROTOCOL

Setting	Value
Seeds	1, 2, 3
Support / class	5
Training / test queries per class	4 / 15
Training pool / class	9 examples
Inner steps / initial rate	1 / 0.01
Outer optimizer / rate	Adam / 0.001
Gradient-norm cap	10
Iterations / meta-batch	100 / 1 task
Test episodes / seed	30
Compute	CPU; 2 threads

The learning rate of the outer optimizer, query count, normalization policy and gradient clipping are explicit implementation settings where the article is incomplete or this practical implementation adds numerical safeguards. The final iteration is always used. Test accuracy does not select a checkpoint, learning rate, seed, or training length. Training curves show adapted query performance on the meta-training pool, not on unseen test classes.

The CPU budget was selected using debug runtime before viewing the reported results. It uses 100 total training tasks per run, versus 37,500 task presentations implied by the paper's 1,500 iterations and 25-task batches. This is a major limitation, even when a resulting accuracy is high. A command in the README restores those published iteration and meta-batch settings while keeping this project's declared preprocessing and evaluation assumptions.

B. Debugging and Verification

Before real training, both way settings passed real-data debug checks: balanced task labels, disjoint support/query sampling, disjoint class and source-file splits, finite gradients, a nonzero difference between full and first-order meta-gradients, unchanged initialization after task adaptation, and decreasing loss when optimizing a repeated training task. Debug weights are discarded. The repeated-task exercise is an optimization check, not a generalization result.

The final verification additionally recomputes every saved accuracy, macro precision, macro recall, F1 score and confusion matrix from the per-query prediction files. Source-file hashes are compared with the downloaded files. The first two evaluation episodes of the 3-way and 5-way seed-1 checkpoints reproduce their saved predictions exactly. The notebook executes all 21 code cells with zero error outputs. These checks verify consistency and execution; they do not replace evaluation on independent bearings.

C. Evaluation Measures

Accuracy is the fraction of correctly classified query examples. Precision, recall and F1 are macro-averaged over physical test classes after mapping episode-local labels back to global class IDs. Each episode balances query counts. Reported accuracy mean and sample standard deviation are computed across three seed-level accuracies using ddof=1. An episode-level standard deviation is saved separately and must not be substituted for seed variation. No confidence interval is claimed from dependent, repeatedly sampled windows.

$$\bar{a} = \frac{1}{R}\sum_{r=1}^{R} a_r, \quad s_a = \sqrt{\frac{\sum_{r=1}^{R}(a_r - \bar{a})^2}{R-1}} \tag{6}$$

Here R=3 and a_r is a seed-level accuracy in percent. Macro precision and F1 are calculated per seed over the fixed physical class labels and then averaged over seeds. With equal query counts per class, macro recall equals accuracy. Counts pooled across episodes and seeds are used only to visualize the confusion matrices; they do not supply additional independent training seeds.

D. Three-Way and Five-Way Five-Shot Results

TABLE IV
PER-SEED TEST METRICS (PERCENT)

N	Seed	Acc.	Prec.	Recall	F1
3	1	95.33	95.87	95.33	95.31
3	2	93.85	94.72	93.85	93.80
3	3	92.96	93.18	92.96	92.95
5	1	58.67	61.86	58.67	59.16
5	2	52.44	62.61	52.44	48.37
5	3	62.98	68.55	62.98	59.26

Mean accuracy ± sample SD: 3-way 5-shot: 94.05 ± 1.20%; 5-way 5-shot: 58.03 ± 5.30%. All six seeds are retained. There are 1,350 query predictions per 3-way run and 2,250 per 5-way run. The prediction CSV files allow every metric and confusion matrix to be recalculated. The optional 3-way 1-shot experiment was not run; no result for it is implied.

TABLE V
MEAN AND SAMPLE SD ACROSS THREE SEEDS (PERCENT)

Metric	3-way 5-shot	5-way 5-shot
Accuracy	94.05 ± 1.20	58.03 ± 5.30
Macro precision	94.59 ± 1.35	64.34 ± 3.66
Macro recall	94.05 ± 1.20	58.03 ± 5.30
Macro F1	94.02 ± 1.20	55.60 ± 6.26

Tables IV and V show that the three-way setting is substantially more successful under this profile than the five-way setting. The difference cannot be attributed solely to the number of classes: the held-out class composition and meta-training pool also change. The five-way F1 values further indicate that good aggregate precision for some classes does not imply balanced recovery of all five classes.

E. Training Dynamics

Figs. 1 and 2 plot adapted query loss and accuracy during meta-training. Raw values are shown faintly, with a 10-iteration moving average overlaid in the same seed color. Loss generally decreases and accuracy increases within the small training pools. These curves measure training-task adaptation. In particular, high final training accuracy in the five-way runs coexists with substantially lower unseen-class accuracy; the two quantities must not be conflated.

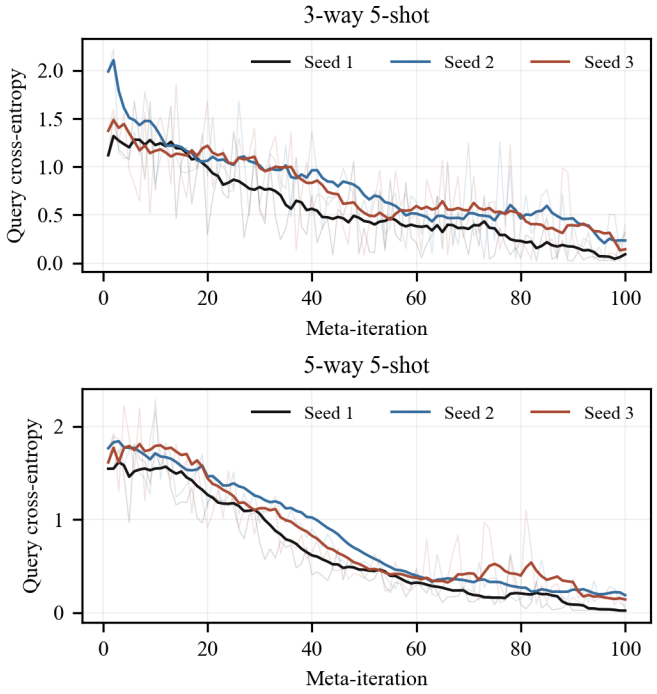


Fig. 1. Adapted meta-training query loss. Top: 3-way; bottom: 5-way. Curves show all three seeds.

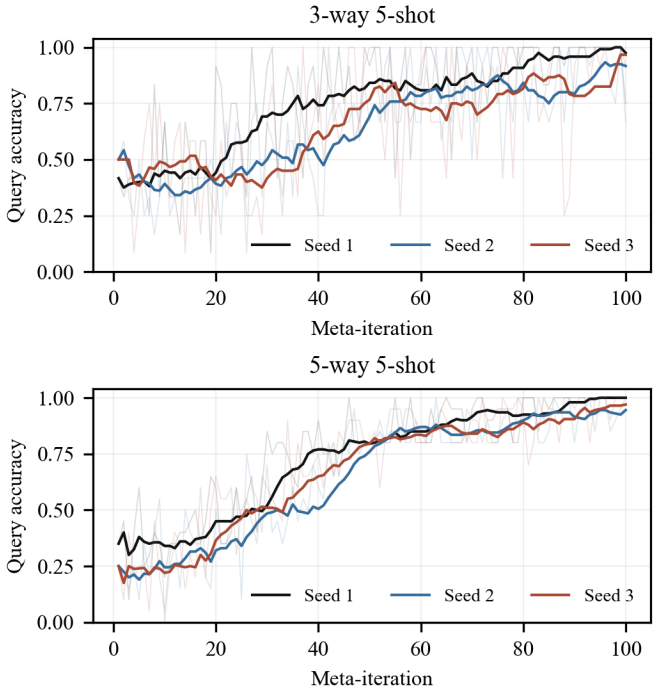


Fig. 2. Adapted meta-training query accuracy. These are training curves, not held-out test curves.

F. Comparison with the Reference Paper

TABLE VI
CONTEXTUAL COMPARISON WITH TABLE V OF [1]

Task	Paper (%)	Ours (%)	Δ (pp)
3-way	90.36 ± 1.38	94.05 ± 1.20	+3.69
5-way	79.98 ± 1.17	58.03 ± 5.30	−21.95

Published reference values are from Table V, learnable-rate MAML [1]. The difference is ours minus the paper, in percentage points; the published results were never used as substitute measurements. A positive gap does not demonstrate superiority, and a negative gap does not isolate a failure of MAML. Training task budget, window construction, temporal support/query separation, normalization and repetition schemes are not controlled to be identical.

The article reports 1,980 segments per class while this project uses non-overlapping windows from an explicit set of recordings. Its normalization and query sampling details are incomplete. Our support-only batch statistics may also behave differently from a transductive implementation. The class label ordering and nine-example meta-training pool are matched, but matching these elements is insufficient to attribute numerical differences to one cause.

The respective comparison gaps are +3.69 and −21.95 percentage points. Neither value is adjusted to improve agreement with the reference. A matched-budget, matched-window comparison would be necessary before attributing these gaps to the learning algorithm. The fixed-rate variant is implemented and its gradients were checked, but it was not run as an additional benchmark; the reference column therefore uses learnable-rate MAML consistently.

G. Confusion Matrix Analysis

The pooled 3-way matrix combines all three seeds using stable physical labels. Per-class recall is OR_007: 91.93%; OR_014: 100.00%; OR_021: 90.22%. The classes differ by outer-race defect diameter; this task evaluates separation among those three severities after adaptation from five examples each.

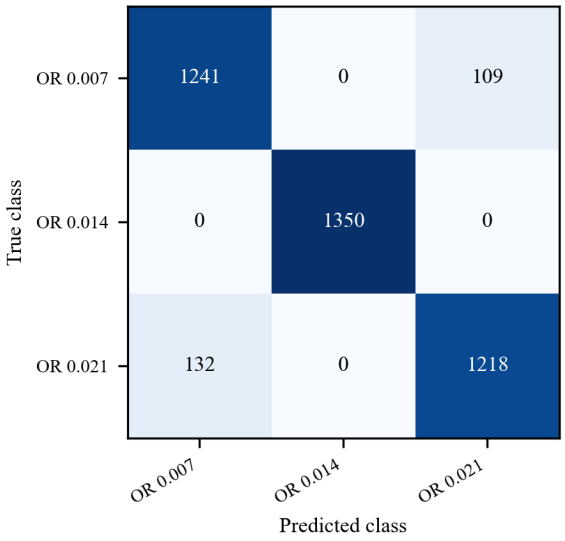


Fig. 3. Three-way confusion matrix over 4,050 query predictions. Labels denote outer-race diameter in inches.

For 5-way evaluation the pooled per-class recalls are IR_014: 25.78%; IR_021: 34.52%; OR_007: 56.67%; OR_014: 95.19%; OR_021: 78.00%. The largest off-diagonal

count is 812, from IR_021 to OR_021. This describes the observed errors; it does not by itself establish a physical mechanism for the confusion. The five-class experiment changes both the number of labels and their composition relative to the three-class experiment.

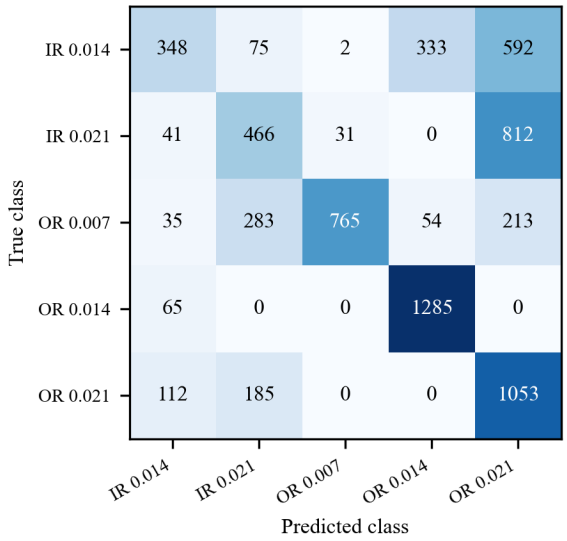


Fig. 4. Five-way confusion matrix over 6,750 query predictions. IR: inner race; OR: outer race. Diameters are in inches.

The five-way matrix shows especially low recall for the two held-out inner-race classes. A substantial number of their queries are assigned to outer-race classes. This is a descriptive finding from the pooled predictions. No additional signal analysis was performed to distinguish physical similarity from limited training, feature bias or normalization effects as the cause of these errors.

H. Computational Cost and Limitations

Measured training time across the six runs was 12.54 minutes; final evaluation time totaled 30.59 minutes. These measurements exclude interpreter startup, downloads, debug verification, plotting and notebook rendering. Exact Python/PyTorch versions and per-run timing are stored in the metric JSON files.

The short schedule is the principal computational limitation. Only three training seeds and 30 episodes per seed were feasible in the submission profile. The same finite windows recur across episodes; source recordings are not independent physical trials. Class-disjoint testing and non-overlapping windows prevent direct sample leakage, but do not remove dependence within a bearing or laboratory rig. The experiment does not measure generalization to a new machine, new bearing specimen, field sensor, unseen speed distribution or naturally developed damage.

There is no independent validation-class pool, so the project uses a predetermined final checkpoint and does not tune hyperparameters on test performance. Normalization, padding, head structure, healthy-signal sample rate and query policy are assumptions. We did not benchmark the fixed-rate variant, conventional supervised learning, or training from a fresh random initialization on the same test episodes. Consequently, these runs alone do not quantify the causal benefit of meta-learning or learned rates over those alternatives.

The principal threats to comparability are thus both computational and statistical. The reduced number of task presentations limits optimization exposure, whereas correlated windows limit independence of the test observations. The absence of matched supervised and untrained-initialization baselines prevents a causal estimate of the benefit of meta-learning. The absence of Paderborn and naturally evolved fault experiments also prevents transfer of the reference paper's artificial-to-real claims to this submission.

V. CONCLUSION

The requested practical project is operational on real CWRU recordings, with measured accuracy 3-way 5-shot: 94.05 ± 1.20%; 5-way 5-shot: 58.03 ± 5.30%. The project reproduces the core computational sequence: episodic sampling, support-set adaptation, query loss, full meta-gradients and an outer optimizer update. The CNN retains the article's four-block architecture, and both requested classification settings have been evaluated under independent training seeds. Saved predictions and source manifests provide traceability from raw recording to reported aggregate.

The appropriate conclusion is methodological and limited: a compact from-scratch MAML implementation can be trained and evaluated on this bearing benchmark under a practical CPU budget. Exact agreement with the original study is not established. Stronger evidence would require resolving the original window construction and normalization, matching training task exposure and repetitions, adding matched baselines, and then testing on independent bearings or an external dataset. Those are extensions rather than completed results in this submission.

The submission provides a self-contained notebook with PyTorch code, dataset instructions, six checkpoints, split manifests, predictions and generated tables. These artifacts support inspection and rerunning of the reported study. Future work should first resolve the reference preprocessing details and match its training budget, then evaluate matched baselines and independent-bearing or external-dataset generalization. These extensions have not been performed here.

REFERENCES

[1] S. Zhang, F. Ye, B. Wang, and T. G. Habetler, "Few-Shot Bearing Fault Diagnosis Based on Model-Agnostic Meta-Learning," arXiv:2007.12851v4, 2021. [Online]. Available: https://arxiv.org/abs/2007.12851

[2] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study," Mechanical Systems and Signal Processing, vol. 64–65, pp. 100–131, 2015. doi: 10.1016/j.ymssp.2015.04.021. https://doi.org/10.1016/j.ymssp.2015.04.021

[3] J. Snell, K. Swersky, and R. S. Zemel, “Prototypical Networks for Few-shot Learning,” in Advances in Neural Information Processing Systems, vol. 30, 2017. https://arxiv.org/abs/1703.05175

[4] A. Antoniou, H. Edwards, and A. Storkey, “How to train your MAML,” in Proc. Int. Conf. Learning Representations, 2019. https://arxiv.org/abs/1810.09502

[5] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” in Proc. Int. Conf. Machine Learning, 2017. [Online]. Available: https://proceedings.mlr.press/v70/finn17a.html

[6] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. Int. Conf. Learning Representations, 2015. https://arxiv.org/abs/1412.6980

[7] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems, vol. 32, 2019. https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library

[8] Z. Li, F. Zhou, F. Chen, and H. Li, “Meta-SGD: Learning to Learn Quickly for Few-Shot Learning,” arXiv:1707.09835, 2017. https://arxiv.org/abs/1707.09835

[9] Case Western Reserve University, “Download a Data File,” Bearing Data Center. Accessed: Oct. 7, 2026. [Online]. Available: https://engineering.case.edu/bearingdatacenter/download-data-file

[10] Case Western Reserve University, “12k Drive End Bearing Fault Data,” Bearing Data Center. Accessed: Oct. 7, 2026. [Online]. Available: https://engineering.case.edu/bearingdatacenter/12k-drive-end-bearing-fault-data

[11] Case Western Reserve University, “Normal Baseline Data,” Bearing Data Center. Accessed: Oct. 7, 2026. [Online]. Available: https://engineering.case.edu/bearingdatacenter/normal-baseline-data

[12] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in Proc. Int. Conf. Machine Learning, vol. 37, pp. 448–456, 2015. https://proceedings.mlr.press/v37/ioffe15.html